

Serviço Nacional de Aprendizagem Comercial
Centro Universitário Senac-RS
Curso Superior de Tecnologia em Redes de Computadores



JAMES JOSÉ GONÇALVES

PRÁTICA DE JOGOS EM REDE E ANÁLISE DE TRÁFEGO

Pelotas
2025

RESUMO

Nesse trabalho eu configurei servidores de jogos em rede usando máquinas virtuais e analisei o tráfego gerado por eles no Wireshark. Fiz testes com três jogos diferentes (Teeworlds, Armagetron e MAME/Kaillera), tanto no meu home lab quanto em sala com os colegas. O objetivo foi entender na prática como os jogos funcionam em rede, principalmente o uso do protocolo UDP, o padrão para jogos de ação. Também comparei o comportamento dos jogos sem restrição de rede e em um ambiente real com vários jogadores, observando coisas como taxa de pacotes, tamanho médio, vazão e conteúdo dos pacotes. No final, consegui visualizar claramente a diferença entre jogar isolado e jogar em um cenário com vários clientes, além de reforçar os conceitos de latência, perda de pacotes e sincronização dos jogos.

Sumário

1. INTRODUÇÃO.....	3
2. DESENVOLVIMENTO.....	4
2.1 Conceitos Fundamentais.....	4
2.2 Configuração do Ambiente.....	5
2.3 Testes com Jogos em Rede.....	5
2.3.1 Teeworlds.....	5
2.3.2 Armagetron.....	8
2.3.3 Jogos Arcade via MAME e Kaillera.....	12
2.4 Análises do Tráfego em Home Lab (Exemplo com Teeworlds).....	15
2.5 Análises do Tráfego na Aula em Grupo (Cenário Real).....	18
2.6 Análise sobre Restrições de Rede (Latência e Perda de Pacotes).....	23
3. CONCLUSÃO.....	24
4. REFERÊNCIAS.....	25

1. INTRODUÇÃO

Os jogos em rede têm um funcionamento bem diferente de outras aplicações, porque tudo precisa acontecer em tempo real. Pequenos atrasos, perdas de pacotes ou qualquer instabilidade já afeta a jogabilidade. Por isso, essa prática foi importante para eu testar de verdade como esses jogos trocam informações na rede, quais protocolos usam, como o servidor responde e como o cliente se comporta.

O roteiro sugeria vários jogos e ferramentas, mas eu adaptei à minha rotina e foquei em três que consegui configurar totalmente: Teeworlds, Armagetron e alguns arcades via MAME usando Kaillera. Configurei servidores e clientes em máquinas virtuais em modo bridge, o que me permitiu reproduzir uma rede LAN funcional. Depois capturei o tráfego no Wireshark e analisei o comportamento de cada jogo, tanto sozinho quanto em grupo durante a aula. Com isso, consegui entender muito melhor como o UDP funciona em jogos rápidos e como a quantidade de jogadores muda completamente o tráfego.

2. DESENVOLVIMENTO

2.1 Conceitos Fundamentais

Antes de iniciar a prática, busquei compreender os conceitos fundamentais que diferenciam o tráfego de jogos de outras aplicações de rede, baseando-me no material teórico da disciplina. Entendi que, para um jogo funcionar bem em rede, a arquitetura e o protocolo de transporte são bem importantes.

A maioria dos jogos modernos e os que testei na prática (como Teeworlds e Armagetron) utilizam a arquitetura Cliente-Servidor. Neste modelo, ele é responsável por processar a lógica do jogo, calcular a física e manter o estado atual da partida (onde cada jogador está, pontuação, etc.). O meu computador (Cliente) funciona apenas como uma interface de entrada e saída: ele envia os comandos (ex: "virei para a esquerda") e recebe do servidor a posição atualizada de todos os elementos para desenhar na tela.

Um dos pontos que achei interessante foi a escolha do protocolo de transporte:

- UDP (User Datagram Protocol): É o padrão para jogos de ação rápida (FPS, Corrida). Como o UDP não garante a entrega e nem a ordem dos pacotes, ele é mais rápido. Em um jogo de tiro, se um pacote com a posição de um jogador se perde, não faz sentido o TCP retransmiti-lo, pois aquela posição já é passado. O jogo prefere receber a nova posição atualizada no próximo pacote instantaneamente.
- TCP (Transmission Control Protocol): É utilizado em jogos onde a integridade dos dados é mais importante que a velocidade, como jogos de turno, estratégia ou cartas (ex: Vitetris/Jogo da Velha). Aqui, garantir que o movimento chegue é o que importa.

2.2 Configuração do Ambiente

Para realizar a prática sozinho, configurei duas máquinas virtuais (VMs) idênticas usando o VMware. Baixei a VM "Win7_Games" do link fornecido no roteiro da prática e copiei o arquivo da VM para criar uma segunda instância. Mudei o nome da segunda VM para "Win7_Games_Cliente" para diferenciar, e também alterei o endereço MAC da placa de rede em cada uma (nas configurações da VM, na aba Network Adapter, gerei um novo MAC) para evitar conflitos na rede. Ambas as VMs estavam configuradas em modo bridge para simular uma rede LAN local, o que permite que elas se comuniquem diretamente como se fossem máquinas físicas na mesma rede, pegando IPs do roteador do host. Isso facilitou testar os jogos em multiplayer LAN sem precisar de colegas, já que em bridge elas veem uma à outra na rede real.

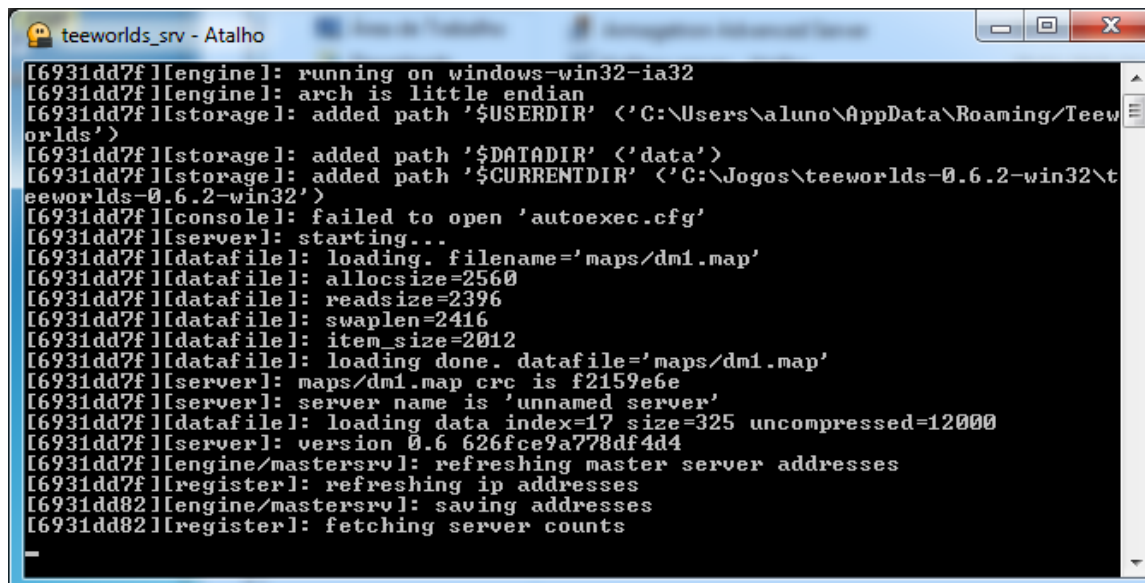
Para capturar o tráfego, iniciei o Wireshark, selecionei a interface de rede Ethernet e comecei a captura antes de iniciar cada jogo em Multiplayer Lan. Parei a captura após jogar um pouco e salvei os arquivos .pcap para análise posterior. Testei três jogos principais: Teeworlds, Armagetron Advanced e alguns emuladores arcade via MAME com Kaillera (como Raiden, Sunset Riders e Double Dragon). Escolhi esses porque atendem aos objetivos da prática: implementar servidores, conectar clientes e analisar tráfego UDP/TCP. Não segui a ordem exata do roteiro, mas adaptei para cobrir os pontos chave, como emuladores (MAME).

2.3 Testes com Jogos em Rede

Comecei testando os jogos em modo LAN multiplayer, iniciando o servidor em uma VM e conectando o cliente na outra. Para cada jogo, descrevo o processo, as teclas de controles e observações durante a execução.

2.3.1 Teeworlds

Iniciei o servidor na VM "Win7_Games_Servidor" executando o arquivo "teeworlds_srv.exe" no prompt de comando. Ele rodou na porta padrão 8303 UDP.



```

teeworlds_srv - Atalho
[6931dd7f][engine]: running on windows-win32-ia32
[6931dd7f][engine]: arch is little endian
[6931dd7f][storage]: added path '$USERDIR' <'C:\Users\aluno\AppData\Roaming\Teew
orlds'>
[6931dd7f][storage]: added path '$DATADIR' <'data'>
[6931dd7f][storage]: added path '$CURRENTDIR' <'C:\Jogos\teeworlds-0.6.2-win32\t
eeworlds-0.6.2-win32'>
[6931dd7f][console]: failed to open 'autoexec.cfg'
[6931dd7f][server]: starting...
[6931dd7f][datafile]: loading. filename='maps/dm1.map'
[6931dd7f][datafile]: allocsize=2560
[6931dd7f][datafile]: readsize=2396
[6931dd7f][datafile]: swaplen=2416
[6931dd7f][datafile]: item_size=2012
[6931dd7f][datafile]: loading done. datafile='maps/dm1.map'
[6931dd7f][server]: maps/dm1.map crc is f2159e6e
[6931dd7f][server]: server name is 'unnamed server'
[6931dd7f][datafile]: loading data index=17 size=325 uncompressed=12000
[6931dd7f][server]: version 0.6 626fce9a778df4d4
[6931dd7f][engine/mastersrv]: refreshing master server addresses
[6931dd82][register]: refreshing ip addresses
[6931dd82][engine/mastersrv]: saving addresses
[6931dd82][register]: fetching server counts

```

Figura 1: Console do servidor Teeworlds

Na VM cliente, abri o jogo Teeworlds, fui para o menu "Multiplayer" > "LAN Multiplayer" > "Network Setup", e vi o servidor listado como "unnamed server". Conectei clicando nele e entrei na partida. O jogo é um platformer 2D multiplayer onde a gente controla um "tee" (personagem redondo fofo) que pula, atira e coleta power-ups em mapas com plataformas.

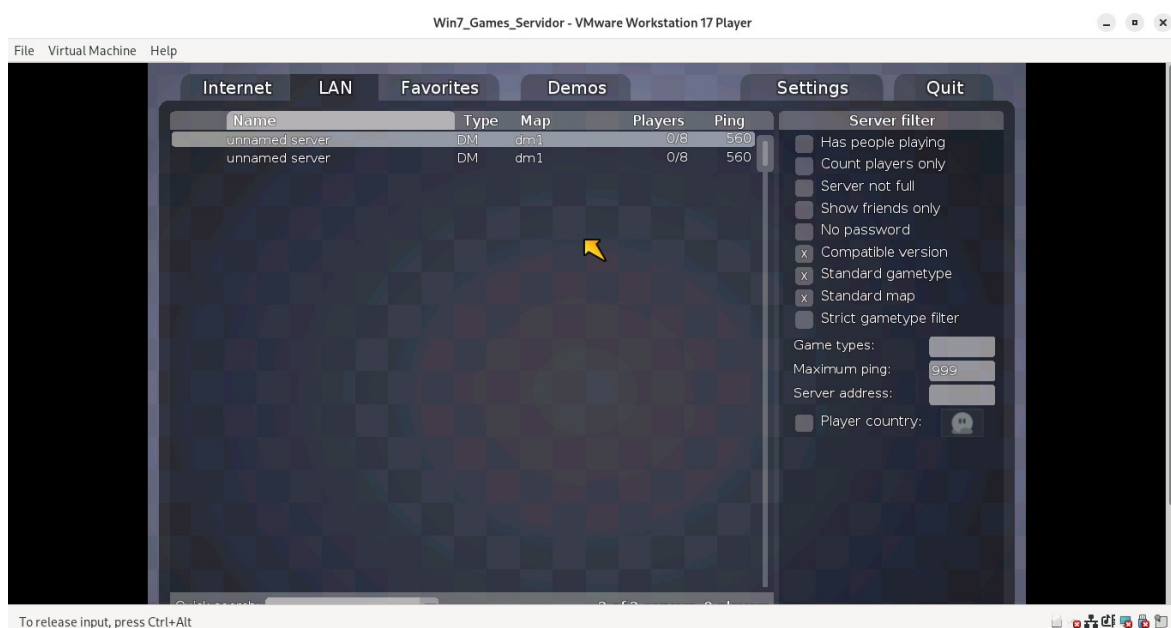


Figura 2: Menu de multiplayer no Teeworlds com servidor LAN listado.

Teclas para jogar:

- Movimentação: A para esquerda, D para direita, W para pular.
- Ataque: Clique esquerdo do mouse para atirar arma principal, clique direito para hook(gancho para se pendurar ou puxar inimigos).
- Chat: T ou Enter para abrir chat.
- Outros: Tab para ver scoreboard, Esc para menu/pause, 1-5 para trocar armas.

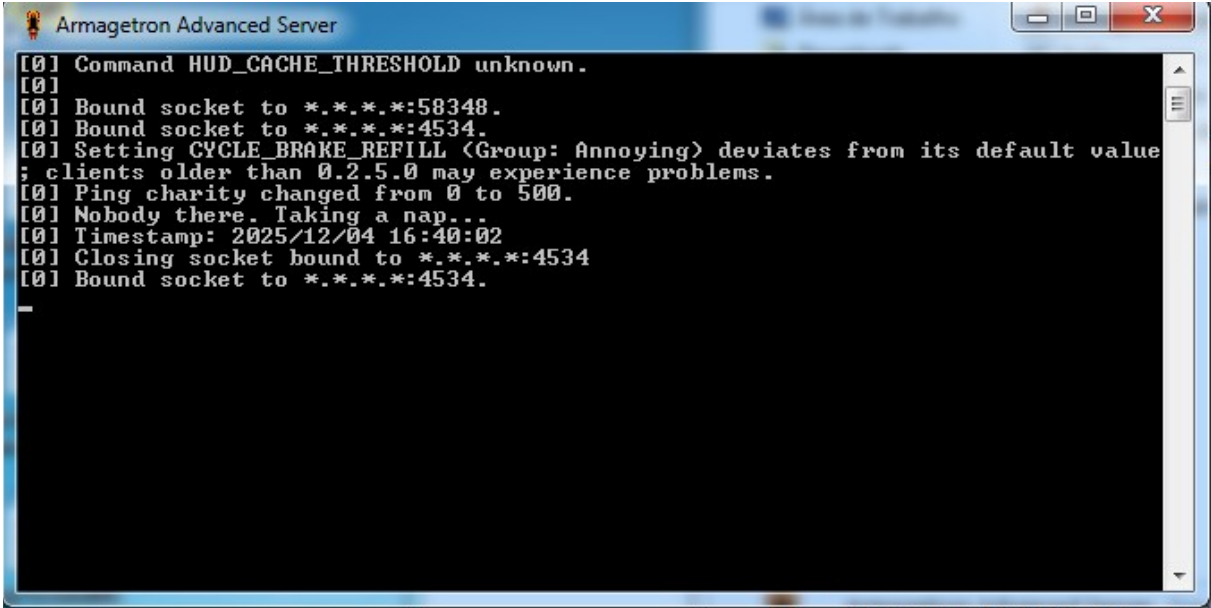
Joguei uma partida rápida, controlando dois tees (um em cada VM), atirando e pulando por plataformas. No print, dá para ver o mapa com personagens, corações de vida, balas e timer, com elementos como árvores e nuvens. Notei que o jogo usa UDP para updates rápidos de posição, e em LAN o lag é zero. Capturei o tráfego no Wireshark durante a partida.



Figura 3: Partida em andamento no Teeworlds

2.3.2 Armagetron

Para esse, iniciei o servidor na VM servidor executando o "Armagetron Advanced Server". A porta padrão é 4534 UDP.

A screenshot of a Windows-style window titled "Armagetron Advanced Server". The window contains a black console area with white text showing the server's startup sequence. The text includes messages about command parsing, socket binding to port 4534, configuration settings, and a timestamp of 2025/12/04 16:40:02.

```
[0] Command HUD_CACHE_THRESHOLD unknown.
[0]
[0] Bound socket to *.*.*.:58348.
[0] Bound socket to *.*.*.:4534.
[0] Setting CYCLE_BRAKE_REFILL <Group: Annoying> deviates from its default value
; clients older than 0.2.5.0 may experience problems.
[0] Ping charity changed from 0 to 500.
[0] Nobody there. Taking a nap...
[0] Timestamp: 2025/12/04 16:40:02
[0] Closing socket bound to *.*.*.:4534
[0] Bound socket to *.*.*.:4534.
```

Figura 4: Console do servidor Armagetron Advanced inicializando na porta 4534

No cliente abri o Armagetron, fui para o menu principal "Play Game", "Player Setup", "System Setup", depois "Multiplayer" > "LAN Multiplayer" e vi o servidor listado como "Host Game Unnamed Server". Conectei e entrei no jogo, que é baseado em Tron: motos deixam rastros de parede, e a gente vence fazendo o oponente bater.



Figura 5: Menu do Armagetron com a opção LAN

Browser de servidores LAN no Armagetron mostrando o servidor local.



Figura 6: server browser no Armagetron

Tecclas para jogar:

- Movimentação: Seta esquerda para virar esquerda, seta direita para virar direita.
- Acelerar/Frear: Seta cima para acelerar/boost, seta baixo para frear.
- Chat: Enter ou / para comandos.
- Outros: V para brake (freio), Tab para scores, Esc para menu.

Joguei rodadas na opção LAN Multiplayer. O jogo sincroniza via UDP, fluido em LAN. Capturei tráfego, vendo pacotes UDP na 4534.

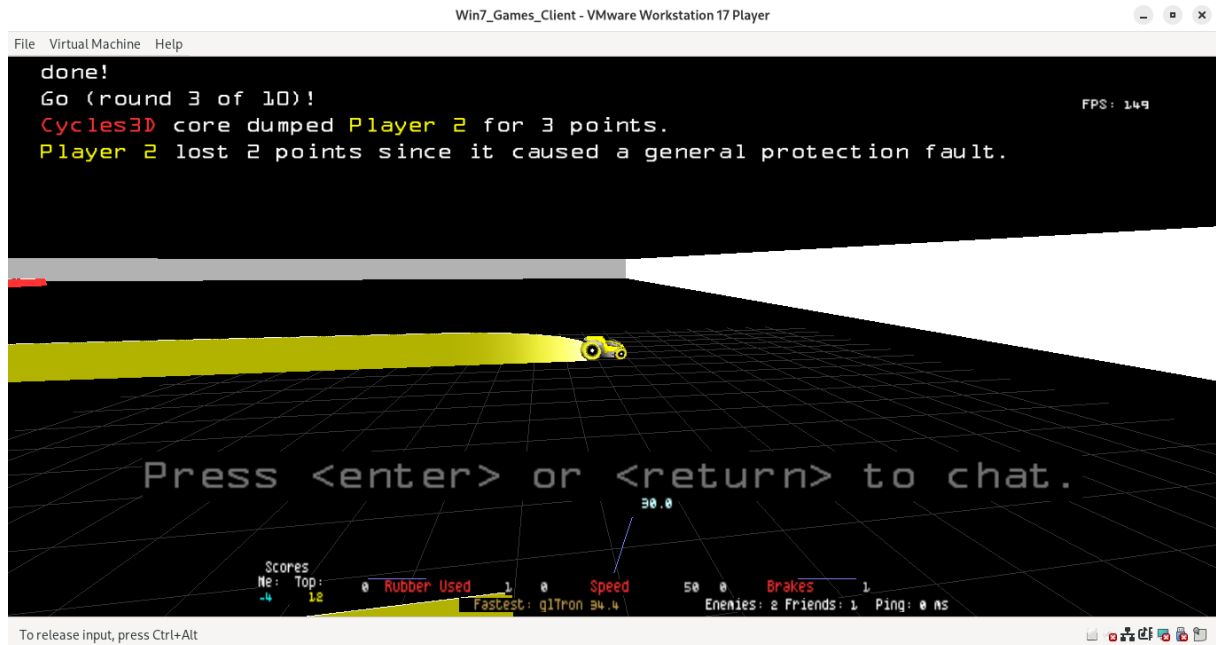


Figura 7: Partida no Armagetron

Outra rodada no Armagetron.

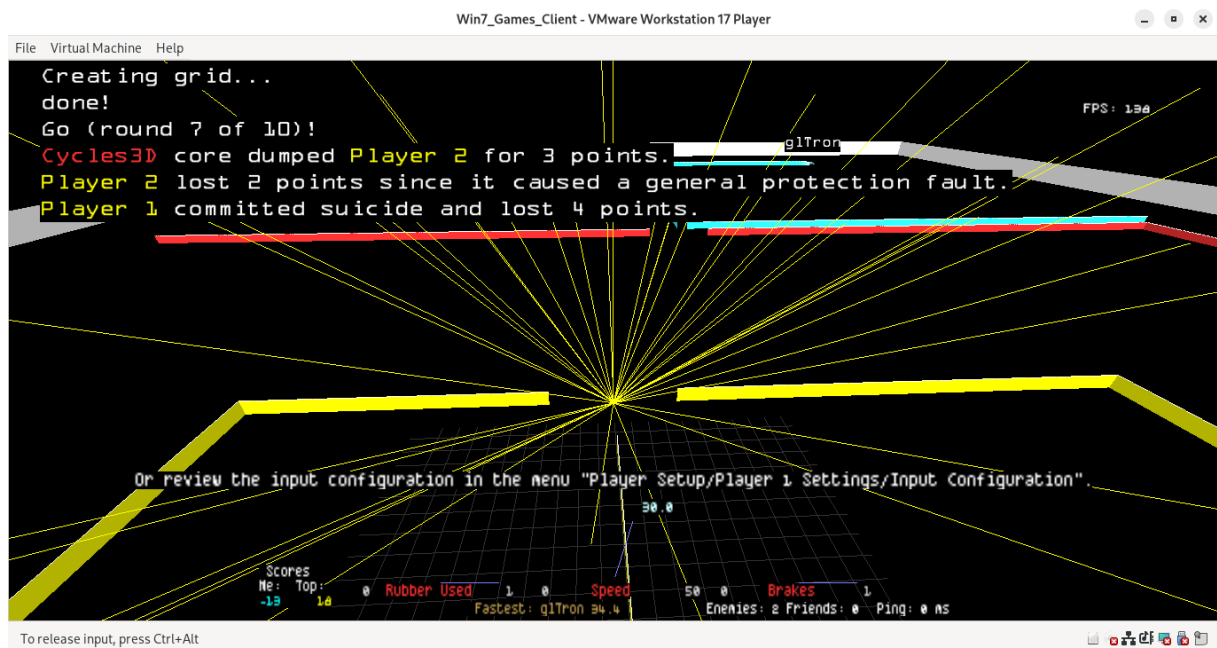


Figura 8: outra partida

2.3.3 Jogos Arcade via MAME e Kaillera

Usei o MAME para emular arcades com multiplayer via Kaillera. Iniciei o Kaillera server na VM servidor com "kaillera-server.exe".

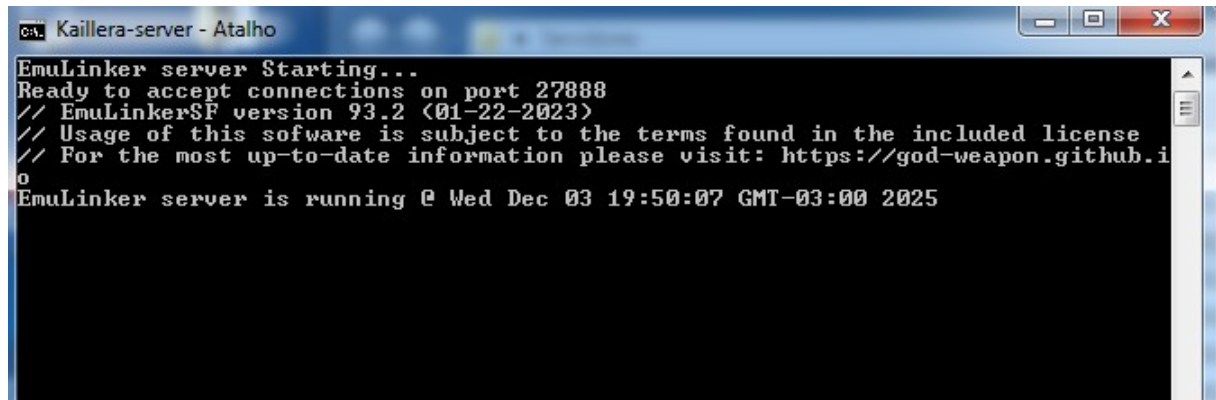


Figura 9: servidor Kaillera

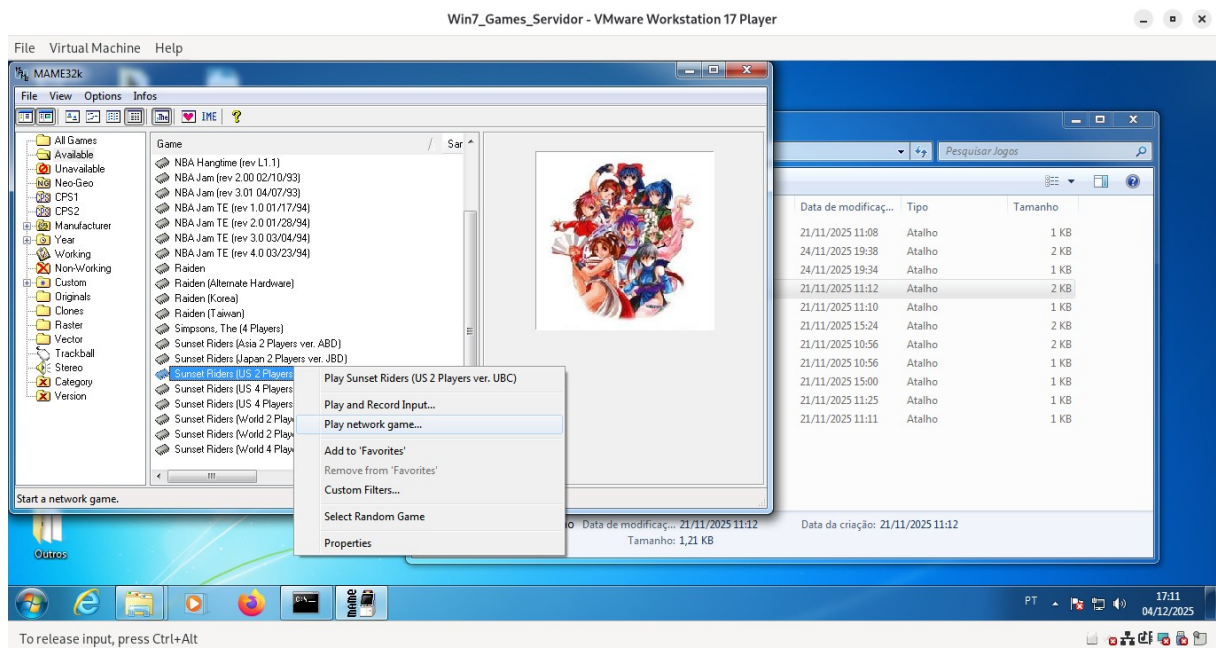


Figura 10: Lista de jogos disponíveis no MAME

Tela mostrando a inicialização do jogo Double Dragon II no MAME.



Figura 11: Double Dragon II no MAME

Executei o jogo Sunset Riders no modo multiplayer pela rede local utilizando o MAME com Kaillera. Cada jogador foi executado em uma VM diferente, e o lobby do MAME mostrou a sessão normalmente. A conexão funcionou sem problemas e o jogo abriu corretamente nas duas máquinas.

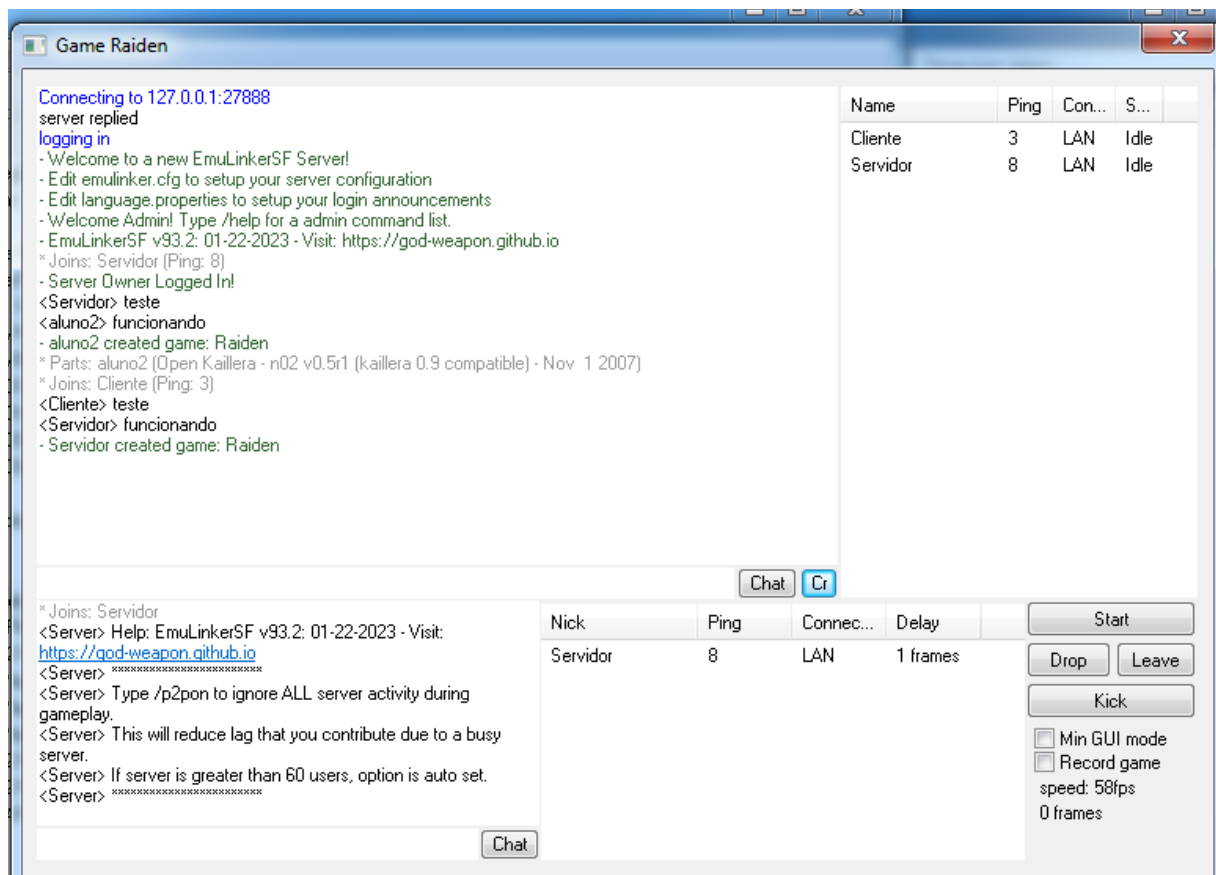


Figura 12: Lobby multiplayer no Raiden

Teclas para jogar:

- Movimentação: Setas (cima/baixo/esquerda/direita) para mover a nave.
- Ataque: Z para tiro principal, X para bomba.

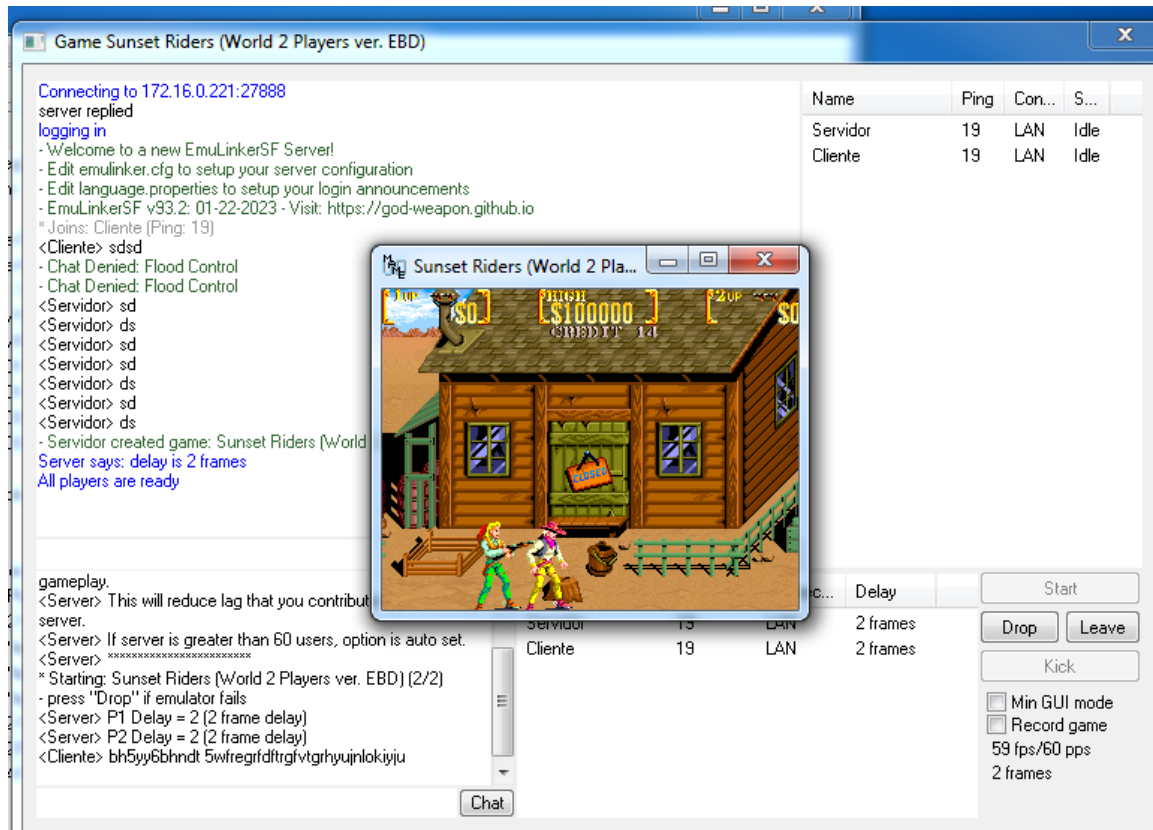


Figura 13: Partida em andamento no Sunset Riders

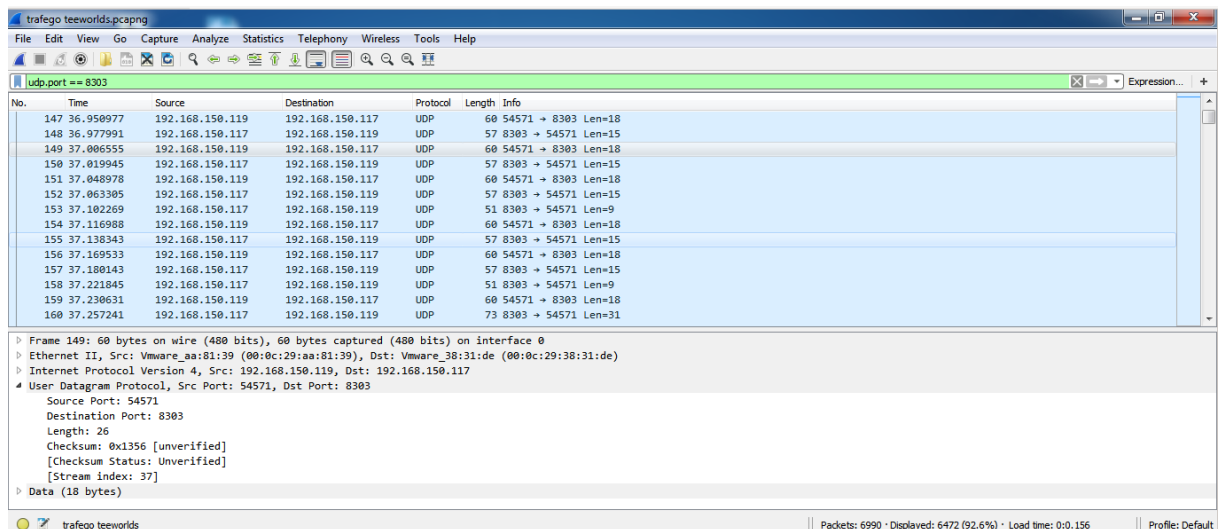
2.4 Análises do Tráfego em Home Lab (Exemplo com Teeworlds)

Fiz a análise do Wireshark no jogo Teeworlds no meu home lab, já que ele usa UDP para ação rápida. Iniciei o servidor em uma VM, conectei o cliente na outra via bridge, joguei uma partida e capturei o tráfego sem restrições.

1. Taxa de Pacotes: A média foi de 38.5 pacotes/s. Isso é significativamente menor do que em uma partida cheia, pois o servidor precisava atualizar apenas o estado de um único cliente.
- Tamanho dos Pacotes: O tamanho médio ficou em 67.5 Bytes. São pacotes muito pequeno, confirmando que o jogo envia apenas o essencial (coordenadas X/Y e inputs) para garantir latência mínima.
 - Vazão: O consumo de banda, com média de 20 kbit/s.

Análise de Conteúdo (Follow UDP Stream): A maior parte do payload é binária e ilegível, o que é esperado para compactar dados de física e movimento. Apesar disso, capturei strings em texto identificando o servidor como "unnamed server" e o jogador como "nameless tee".

Obs: UDP prioriza latência baixa, pacotes pequenos evitam delays, e binário otimiza para multiplayer, com dados mistos (legíveis e criptografados) para privacidade e eficiência.



No.	Time	Source	Destination	Protocol	Length	Info
147	36.950977	192.168.150.119	192.168.150.117	UDP	60	54571 → 8303 Len=18
148	36.977991	192.168.150.117	192.168.150.119	UDP	57	8303 → 54571 Len=15
149	37.006555	192.168.150.119	192.168.150.117	UDP	60	54571 → 8303 Len=18
150	37.019945	192.168.150.117	192.168.150.119	UDP	57	8303 → 54571 Len=15
151	37.040978	192.168.150.119	192.168.150.117	UDP	60	54571 → 8303 Len=18
152	37.063305	192.168.150.117	192.168.150.119	UDP	57	8303 → 54571 Len=15
153	37.102269	192.168.150.117	192.168.150.119	UDP	51	8303 → 54571 Len=9
154	37.116988	192.168.150.119	192.168.150.117	UDP	60	54571 → 8303 Len=18
155	37.138343	192.168.150.117	192.168.150.119	UDP	57	8303 → 54571 Len=15
156	37.169533	192.168.150.119	192.168.150.117	UDP	60	54571 → 8303 Len=18
157	37.180143	192.168.150.117	192.168.150.119	UDP	57	8303 → 54571 Len=15
158	37.221845	192.168.150.117	192.168.150.119	UDP	51	8303 → 54571 Len=9
159	37.230631	192.168.150.119	192.168.150.117	UDP	60	54571 → 8303 Len=18
160	37.257241	192.168.150.117	192.168.150.119	UDP	73	8303 → 54571 Len=31

Frame 149: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface 0
 Ethernet II, Src: Vmware aa:01:39 (00:0c:29:aa:81:39), Dst: Vmware_38:31:de (00:0c:29:38:31:de)
 Internet Protocol Version 4, Src: 192.168.150.119, Dst: 192.168.150.117
 User Datagram Protocol, Src Port: 54571, Dst Port: 8303
 Source Port: 54571
 Destination Port: 8303
 Length: 26
 Checksum: 0x1356 [unverified]
 [Checksum Status: Unverified]
 [Stream Index: 37]
 Data (18 bytes)

trafego teeworlds | Packets: 6990 · Displayed: 6472 (92.6%) · Load time: 0:0.156 | Profile: Default

Figura 14: Lista de pacotes UDP na porta 8303 durante partida no Teeworlds

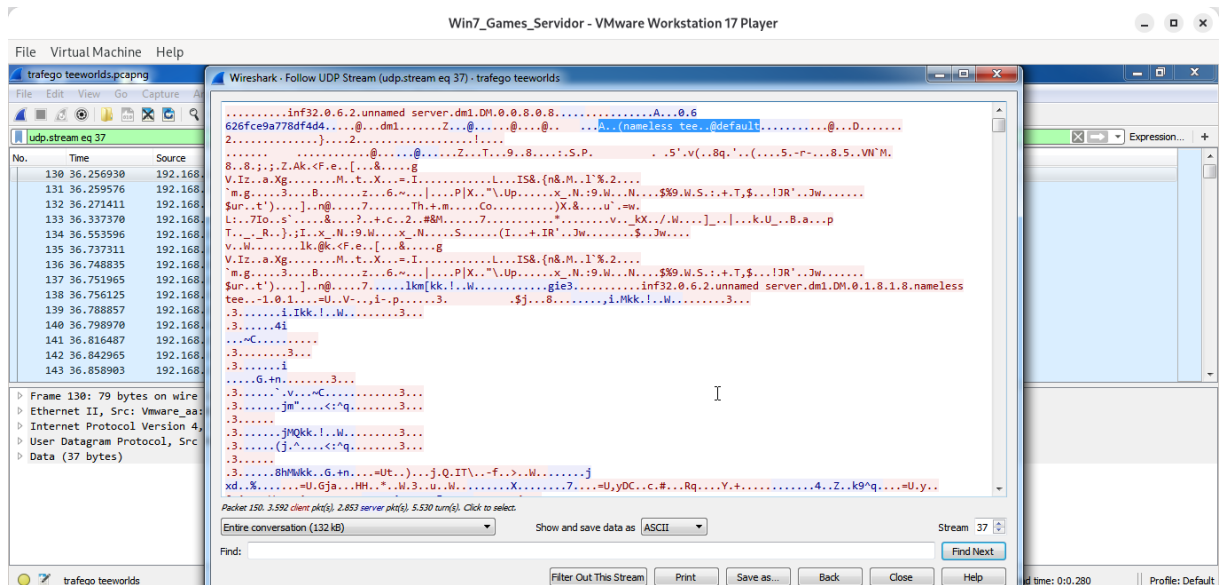


Figura 15: Follow UDP Stream no Teeworlds, com dados binários e strings legíveis como 'nameless tee' e 'unnamed server'

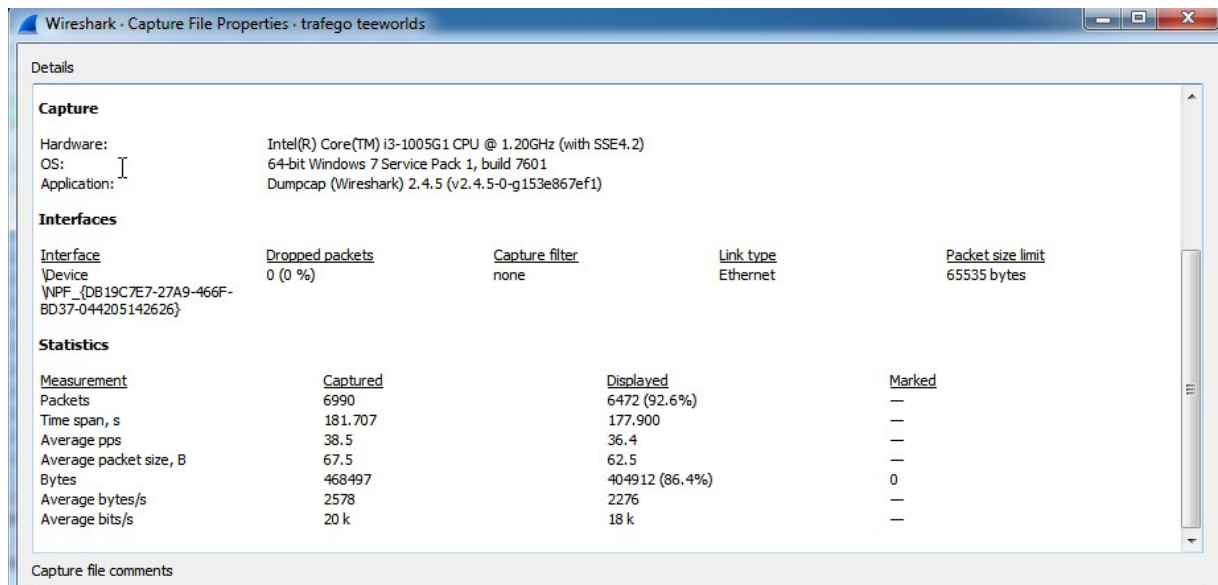


Figura 16: Estatísticas gerais da captura no Teeworlds

2.5 Análises do Tráfego na Aula em Grupo (Cenário Real)

Enquanto no Home Lab observei um tráfego limpo, na aula o cenário mudou drasticamente. Diferente do teste isolado com as VMs, aqui realizamos a conexão em uma rede local (LAN) real com a turma inteira, gerando interações simultâneas em jogos como Teeworlds, Soldat e emuladores via Kaillera (MAME). O objetivo principal desta etapa foi capturar esse tráfego intenso para analisar como a rede se comporta sob estresse real, com múltiplos clientes enviando e recebendo dados ao mesmo tempo.

Ao analisar as capturas no Wireshark, a predominância absoluta é do protocolo UDP. Isso era esperado, pois jogos de ação rápida priorizam a velocidade (transmissão em tempo real) em vez da garantia de entrega (TCP), para evitar "lags" na movimentação.

Utilizando a função Follow UDP Stream, foi possível "ler" o que estava passando dentro dos pacotes:

1. Teeworlds (Porta 8303): A maior parte do conteúdo é binário (caracteres ilegíveis) usados para sincronizar posição e física do jogo. Porém, encontrei strings de texto claro misturadas. Foi possível identificar os nomes dos jogadores presentes na partida, como "Madruga", "Gabriel" e o meu próprio usuário, "James", além de eventos do sistema como "nameless tee entered and joined the game".
2. Kaillera/MAME (Porta 27908): No tráfego do emulador, identifiquei o nome do jogo carregado, "Sunset Riders (US 4 Players ver. UAC)", e nomes de jogadores como "Fred" e "Richard M", além de status de conexão. O padrão aqui sugeriu pacotes menores e constantes, típicos de emulação que precisa sincronizar cada "frame" do jogo de arcade entre os participantes.



Figura 17: Follow UDP Stream no Teeworlds durante a aula



Figura 18: Follow UDP Stream no Kaillera/MAME.

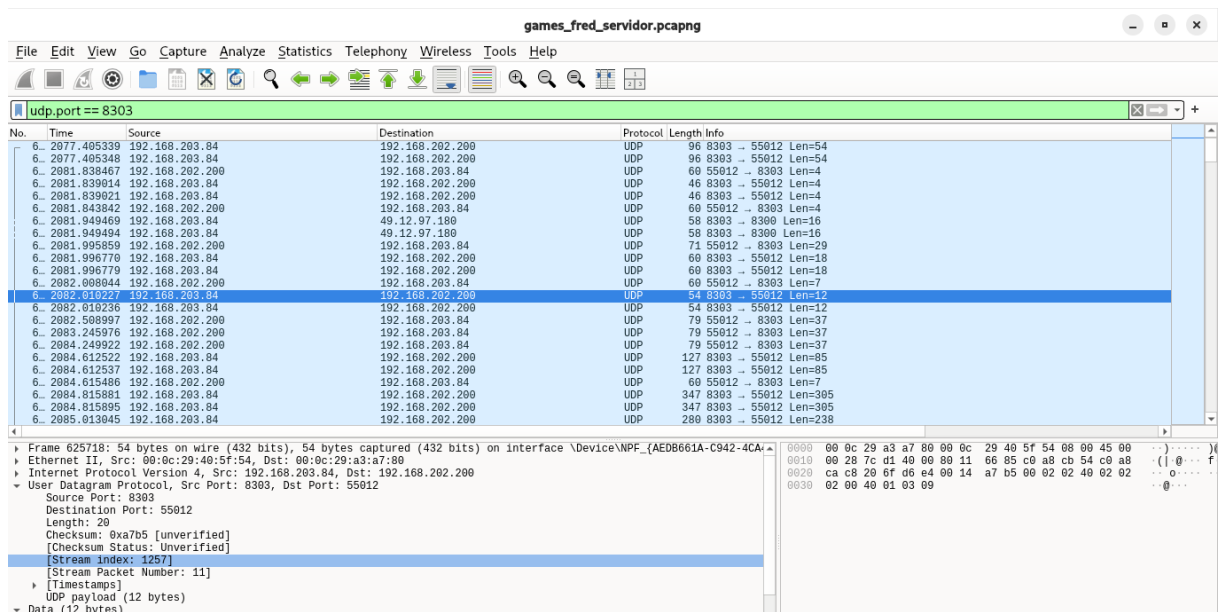


Figura 19: Lista de pacotes na porta 27908 (Kaillera), mostrando a comunicação rápida e repetitiva típica da emulação de arcade em rede.

Análise das Estatísticas: comparei as estatísticas gerais das duas capturas buscando entender o impacto na rede:

- Taxa de Pacotes : A captura (focada no Teeworlds e outros) apresentou uma média de 234.3 pacotes por segundo, um número alto por causa da quantidade de jogadores se movendo e atirando ao mesmo tempo. Já no Kaillera, a média foi de 101.2 pps.
- Tamanho Médio dos Pacotes: No cenário Teeworlds, a média foi de 724 bytes. Esse tamanho maior se justifica pela quantidade de informações extras trocadas (chat, nomes, estados de múltiplos jogadores). No Kaillera, a média foi de apenas 85 bytes, o que faz sentido para emuladores: eles enviam apenas comandos simples.
- Vazão (Throughput): A vazão média no cenário geral foi de aproximadamente 1.3 kbit/s, enquanto no Kaillera ficou em 68 kbit/s. Ambos os valores são baixos para padrões de redes modernas (Gigabit), provando que o gargalo em jogos online geralmente não é a largura de banda (velocidade da internet), mas sim a latência (ping) e a quantidade de pacotes que o processador precisa tratar por segundo.

Wireshark · Capture File Properties · games_fred_servidor.pcapng

Details

Length:

535 MB

Hash (SHA256):

fcc896853619c66c4a823484c530c5fd0a8a67316c87f7b949fb70311d31f017

Hash (SHA1):

c88ff80107fc3b950c6ad3cd6d78c440c0657271

Format:

Wireshark/... - pcapng

Encapsulation:

Ethernet

Time

First packet:

2025-11-25 20:45:20

Last packet:

2025-11-25 21:35:34

Elapsed:

00:50:14

Capture

Hardware:

AMD Ryzen 5 PRO 5650G with Radeon Graphics (with SSE4.2)

OS:

64-bit Windows 11 (24H2), build 26100

Application:

Dumpcap (Wireshark) 4.4.5 (v4.4.5-0-g47253bcf3773)

Interfaces

Interface	Dropped packets	Capture filter	Link type	Packet size limit (snaplen)
Ethernet	0 (0.0%)	none	Ethernet	262144 bytes

Statistics

Measurement	Captured	Displayed	Marked
Packets	706363	54160 (7.7%)	—
Time span, s	3014.177	2997.142	—
Average pps	234.3	18.1	—
Average packet size, B	724	81	—
Bytes	511469651	4383979 (0.9%)	0
Average bytes/s	169 k	1.462	—
Average bits/s	1.357 k	11 k	—

Figura 20: Estatísticas da captura geral (Teeworlds)

Wireshark · Capture File Properties · kaillera_4players_mame_25112025.pcapng

Details

Hash (SHA256):

Hash (SHA1):

Format:

Encapsulation:

Time

First packet:

Last packet:

Elapsed:

Capture

Hardware:

OS:

Application:

Interfaces

Interface	Dropped packets	Capture filter	Link type	Packet size limit (snaplen)
\\Device\NPF_{DB19C7E7-27A9-466F-BD37-044205142626}	0 (0.0%)	none	Ethernet	65535 bytes

Statistics

Measurement	Captured	Displayed	Marked
Packets	26506	26506 (100.0%)	—
Time span, s	261.819	261.819	—
Average pps	101.2	101.2	—
Average packet size, B	85	85	—
Bytes	2249318	2249318 (100.0%)	0
Average bytes/s	8.591	8.591	—
Average bits/s	68 k	68 k	—

Figura 21: Estatísticas da captura específica do Kaillera. Observa-se pacotes bem menores (85 bytes em média) e menor vazão, característico de tráfego de comandos de controle.

2.6 Análise sobre Restrições de Rede (Latência e Perda de Pacotes)

Embora o foco desta prática tenha sido a análise do tráfego em condições funcionais, é fundamental compreender, teoricamente, como o protocolo UDP se comportaria sob condições adversas de rede (simuláveis por ferramentas como WANem ou Clumsy), conforme proposto no roteiro.

1. Inserção de Latência (Delay/Ping Alto):

- Em jogos como Teeworlds (FPS rápido), o aumento da latência não pararia o jogo (pois não há handshake contínuo como no TCP), mas causaria o efeito de atraso entre o comando do jogador (ex: atirar) e a resposta visual. O servidor receberia a ação "tarde demais".
- No Kaillera (Emulação), que sincroniza frames, a latência alta causaria uma "câmera lenta" ou travamentos para todos os jogadores, pois o emulador precisa que todos estejam no mesmo frame para avançar.

2. Perda de Pacotes (Packet Loss):

- Como o UDP não retransmite pacotes, a perda de dados resultaria em "saltos" visuais. Se o pacote com a coordenada "X=10" se perde e o próximo que chega é "X=50", o personagem "teletransporta" na tela do cliente.
- Diferente de um download de arquivo (TCP), onde a perda trava o download até o pacote chegar, no jogo a ação continua, mas com falhas visuais e de física.

3. CONCLUSÃO

Essa prática de jogos em rede foi bem útil pra eu entender na prática como o UDP se comporta e como os jogos trocam informações em tempo real. Ao configurar os servidores e capturar o tráfego no Wireshark, eu consegui enxergar claramente como cada jogo manda pacotes de forma constante e como a resposta precisa ser rápida pra não gerar atraso ou travamentos. O teste sozinho no home lab e depois com vários colegas jogando mostrou na hora a diferença no volume de pacotes e no comportamento da rede, algo que só vendo na prática pra entender. Também ficou mais fácil visualizar como o servidor centraliza a lógica do jogo e como o cliente basicamente só envia comandos e recebe atualizações. No geral, foi uma prática que fechou bem o conteúdo de UDP, latência, perda de pacotes e sincronização, e ajudou a consolidar o que venho estudando sobre redes em aplicações em tempo real.

4. REFERÊNCIAS

- <https://doompdf.pages.dev/doom.pdf> (Jogo DOOM em PDF)
- Material do servidor Mussum
- Notebooklm